

Raritone Backend Database Schema & Module Documentation

MEKA HRISHI TEJA CHOWDARY

Overview

The Raritone backend is built using Node.js, Express.js, MongoDB, and Mongoose. The architecture is designed to support scalable APIs, real-time communication, AI-powered workflows, and optimized media handling.

The backend includes:

- Modular API structure
- Database schemas
- Middleware layers
- Validation systems
- Real-time Socket.IO workflows

1. User Schema

Purpose

The User schema stores authentication and profile information for platform users.

Main Fields

```
{  
  name,  
  email,  
  password,  
  profileImage,  
  role,  
  createdAt  
}
```

Relationships

- Linked with Wishlist
- Linked with Orders
- Linked with Measurements

- Linked with Try-On History

AI Usage

User data can support:

- Personalized recommendations
- AI styling suggestions
- Avatar personalization

2. Product Schema

Purpose

The Product schema stores product catalog information.

Main Fields

```
{  
  title,  
  description,  
  price,  
  category,  
  stock,  
  images,  
  imagePublicIds,  
  createdAt  
}
```

Relationships

- Linked with Wishlist
- Linked with Orders
- Linked with TryOn
- Linked with ProductMapping

AI Usage

Product data supports:

- Recommendation systems

- AI compatibility matching
- Virtual try-on processing

3. Measurement Schema

Purpose

Stores body measurements for personalized fitting and AI compatibility.

Main Fields

```
{  
  userId,  
  height,  
  weight,  
  chest,  
  waist,  
  hip,  
  shoulder,  
  inseam,  
  shoeSize  
}
```

Relationships

- Linked with User schema
- Linked with Avatar schema
- Supports TryOn workflow

AI Usage

Measurement data helps:

- Improve try-on accuracy
- Body compatibility analysis
- Personalized recommendations

4. TryOn Schema

Purpose

Stores virtual try-on requests and processed results.

Main Fields

```
{  
  userId,  
  productId,  
  userImage,  
  processedImage,  
  status,  
  createdAt  
}
```

Relationships

- Linked with User schema
- Linked with Product schema
- Connected with Cloudinary storage

AI Usage

Supports:

- AI try-on processing
- Real-time status updates
- Virtual fitting history

5. Wishlist Schema

Purpose

Stores user wishlist products.

Main Fields

```
{  
  userId,  
  products,  
  createdAt  
}
```

Relationships

- Linked with User schema
- Linked with Product schema

AI Usage

Wishlist data may support:

- Recommendation systems
- Personalized suggestions
- User preference analysis

6. Orders Schema

Purpose

Stores purchase and order history information.

Main Fields

```
{  
  userId,  
  products,  
  totalAmount,  
  paymentStatus,  
  deliveryStatus,  
  createdAt  
}
```

Relationships

- Linked with User schema
- Linked with Product schema

AI Usage

Order history can support:

- Recommendation engines
- Shopping behavior analysis
- Personalized offers

7. Avatar Schema

Purpose

Stores AI-generated avatar information for virtual try-on experiences.

Main Fields

```
{  
  userId,  
  avatarImage,  
  bodyType,  
  skinTone,  
  gender,  
  avatarModel,  
  createdAt  
}
```

Relationships

- Linked with User schema
- Connected with TryOn workflow

AI Usage

Supports:

- AI avatar generation
- Personalized virtual fitting
- Body modeling

8. ProductMapping Schema

Purpose

Stores compatibility information between products and user body types.

Main Fields

```
{  
  productId,  
  category,
```

```
clothType,  
supportedBodyTypes,  
compatibility,  
avatarCompatibility  
}
```

Relationships

- Linked with Product schema
- Connected with Avatar schema

AI Usage

Supports:

- Recommendation engine
- AI compatibility analysis
- Personalized try-on matching

Database Relationship Flow

User



Measurements



Avatar



TryOn



ProductMapping



Products



Orders/Wishlist

Backend Module Documentation

Controllers

Controllers handle:

- API business logic
- Database operations
- Request processing
- Response handling

Examples:

- authController
- productController
- tryOnController
- avatarController

Routes

Routes define API endpoints.

Examples:

- /api/auth
- /api/products
- /api/tryon
- /api/profile
- /api/avatar

Middleware

Middleware handles:

- Authentication
- Validation
- File uploads
- Error handling
- Security

Examples:

- authMiddleware

- `errorMiddleware`
- `uploadMiddleware`

Validators

Validators use Joi for:

- Request validation
- Input sanitization
- Secure API handling

Examples:

- `loginValidator`
- `registerValidator`
- `productValidator`
- `tryOnValidator`

Models

Models define MongoDB schemas and database structure.

Examples:

- `User`
- `Product`
- `Measurement`
- `TryOn`
- `Avatar`
- `ProductMapping`

Schema Verification

Schemas were verified through:

- Backend startup testing
- MongoDB model validation
- API integration testing

Verified:

- Schema exports

- Backend compatibility
- Database consistency

Conclusion

The Raritone backend database architecture provides a scalable and AI-ready foundation for:

- E-commerce workflows
- Virtual try-on systems
- Real-time communication
- AI integrations
- Personalized recommendations

The modular backend structure ensures:

- Maintainability
- Scalability
- Secure API development
- Future AI expansion